

GCC Compilation Process: Step-by-Step Guide

The compilation of a C program involves four main stages: Preprocessing, Compilation, Assembly, and Linking. This document provides a practical demonstration of each phase.

Step 1: Original C Program

- **Objective:** View the original source code.
- **Command:** `cat main.c`
- **Image Placeholder:**

```
root@ec34d732f510:/# echo '#include <stdio.h>
int main() {
    printf("Hello World!\\n");
    return 0;
}' > main.c
cat main.c
#include <stdio.h>
int main() {
    printf("Hello World!\\n");
    return 0;
}
root@ec34d732f510:/# |
```

- **Observation:** Displays the initial C source code written by the programmer.

Step 2: Preprocessing

- **Objective:** Process directives like `#include` and macros to generate the preprocessed source code (`.i`).
- **Command:** `gcc -E main.c -o main.i` then `head -30 main.i`

- **Image Placeholder:**

```
root@ec34d732f510:/# gcc -E main.c -o main.i
root@ec34d732f510:/# head -30 main.i
# 0 "main.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "main.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
# 1 "/usr/include/features-time64.h" 1 3 4
# 20 "/usr/include/features-time64.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 21 "/usr/include/features-time64.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 1 3 4
# 19 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 20 "/usr/include/x86_64-linux-gnu/bits/timesize.h" 2 3 4
# 22 "/usr/include/features-time64.h" 2 3 4
# 432 "/usr/include/features.h" 2 3 4
# 539 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 730 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 731 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 732 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 540 "/usr/include/features.h" 2 3 4
```

- **Observation:** The code expands to include standard library declarations, removing comments and resolving macros.

Step 3: Compilation

- **Objective:** Convert the preprocessed C code into assembly language (.s).
- **Command:** `gcc -S main.i -o main.s` then `head -30 main.s`

- **Image Placeholder:**

```
root@ec34d732f510:/# gcc -S main.i -o main.s
root@ec34d732f510:/# head -30 main.s
        .file     "main.c"
        .text
        .section   .rodata
.LC0:
        .string   "Hello World!\\n"
        .text
        .globl    main
        .type      main, @function
main:
.LFB0:
        .cfi_startproc
        endbr64
        pushq     %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq      %rsp, %rbp
        .cfi_def_cfa_register 6
        leaq      .LC0(%rip), %rax
        movq      %rax, %rdi
        movl      $0, %eax
        call      printf@PLT
        movl      $0, %eax
        popq      %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE0:
        .size      main, .-main
        .ident      "GCC: (Ubuntu 15.2.0-16ubuntu1) 15.2.0"
        .section    .note.GNU-stack,"",@progbits
```

- **Observation:** The compiler translates the C syntax into low-level CPU-specific assembly instructions.

Step 4: Assembly

- **Objective:** Convert the assembly instructions into machine-readable object code (.o) and inspect it.
- **Command:** `gcc -c main.s -o main.o` then `objdump -d main.o`

- **Image Placeholder:**

```
root@ec34d732f510:/# gcc -c main.s -o main.o
root@ec34d732f510:/# objdump -d main.o

main.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <main>:
 0:  f3 0f 1e fa          endbr64
 4:  55                   push    %rbp
 5:  48 89 e5             mov     %rsp,%rbp
 8:  48 8d 05 00 00 00 00 lea     0x0(%rip),%rax      # f <main+0xf>
 f:  48 89 c7             mov     %rax,%rdi
12:  b8 00 00 00 00      mov     $0x0,%eax
17:  e8 00 00 00 00      call    1c <main+0x1c>
1c:  b8 00 00 00 00      mov     $0x0,%eax
21:  5d                   pop     %rbp
22:  c3                   ret
```

- **Observation:** The assembler creates binary machine code. The **objdump** tool allows us to view the raw hex format alongside assembly.

Step 5: Linking

- **Objective:** Combine the object file with required libraries to create the final executable binary.
- **Command:** **gcc main.o -o main** then **file main**
- **Image Placeholder:**

```
root@ec34d732f510:/# gcc main.o -o main
root@ec34d732f510:/# file main
main: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=21234bf3707979b6c1dd8453890f7122d222ab0a, for GNU/Linux 3.2.0, not stripped
root@ec34d732f510:/#
```

- **Observation:** The linker resolves external function calls (like **printf**) and creates the final ready-to-run program.

Step 6: Execution

- **Objective:** Run the final executable program.
- **Command:** **./main**

- **Image Placeholder:**

```
root@ec34d732f510:/# gcc -c main.s -o main.o
root@ec34d732f510:/# objdump -d main.o

main.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <main>:
 0:  f3 0f 1e fa          endbr64
 4:  55                   push    %rbp
 5:  48 89 e5             mov     %rsp,%rbp
 8:  48 8d 05 00 00 00 00 lea     0x0(%rip),%rax      # f <main+0xf>
 f:  48 89 c7             mov     %rax,%rdi
12:  b8 00 00 00 00      mov     $0x0,%eax
17:  e8 00 00 00 00      call    1c <main+0x1c>
1c:  b8 00 00 00 00      mov     $0x0,%eax
21:  5d                   pop     %rbp
22:  c3                   ret
```

- **Observation:** The OS executes the compiled binary, producing the expected output.